

Rosetta Decode — Data Handling & Security Overview

Purpose: Answer the question every prospect asks before uploading a single SAS file — “*where does my code go?*” — with a factual account of processing location, LLM call routing, GDPR posture, retention, and deployment region.

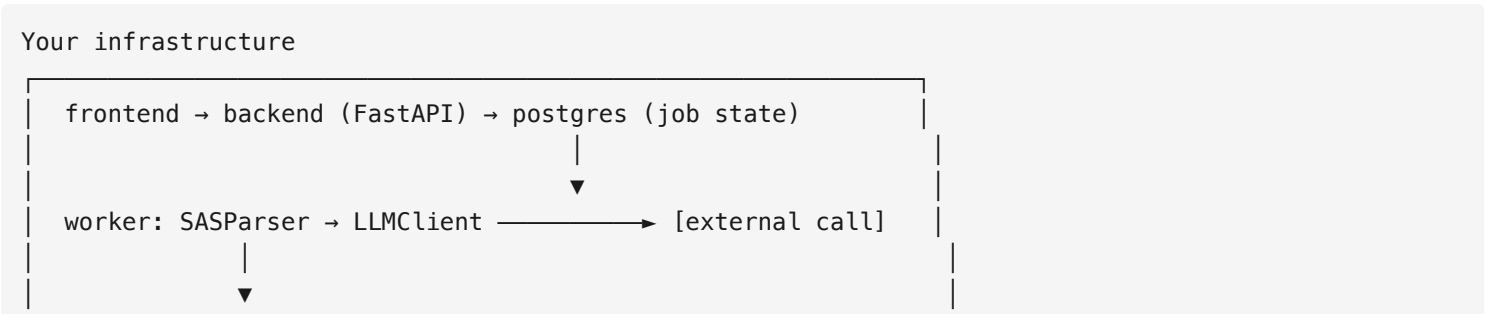
1. Short answer

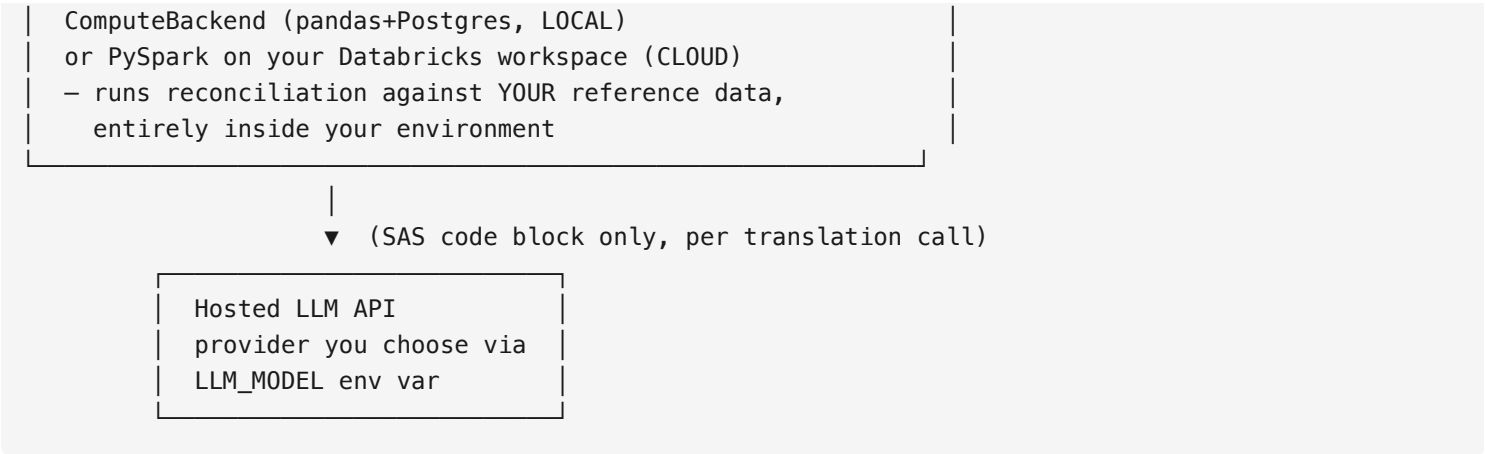
Rosetta Decode is deployed **inside your own infrastructure** (Docker Compose on your servers, or your Azure/Databricks workspace). ADC does not operate a shared, multi-tenant SaaS instance that ingests customer SAS code. The only traffic that leaves your environment is the **SAS code block being translated**, sent directly from your worker service to the LLM provider **you** configure — never routed through ADC systems.

Question	Answer
Where is my SAS code stored?	In your PostgreSQL instance, inside your deployment. Never on ADC infrastructure.
Where is my reference/sample data (CSV, sas7bdat) processed?	Locally, by the ComputeBackend (pandas/PostgreSQL or your Databricks/PySpark cluster). Never sent to the LLM.
What actually goes to the LLM?	Parsed SAS code blocks (source logic) only — for translation. No customer data rows.
Who chooses the LLM provider?	You do, via LLM_MODEL — Anthropic, OpenAI, or Azure OpenAI (for EU data residency).
Who is the data controller?	You remain the data controller at all times — ADC does not process your data as a separate controller.
What region does this run in?	Whatever region your deployment (VM, container host, or Azure/Databricks workspace) is provisioned in.

2. Where processing happens

Rosetta Decode is four Docker images (backend, worker, postgres, frontend), deployed as a unit inside your network boundary — on-prem, in your cloud tenant, or paired with your Databricks workspace.





Reference datasets used for reconciliation (row counts, schema parity, aggregate checks) never leave your ComputeBackend — they are read, transformed, and compared locally or in your own Databricks workspace. The **only** outbound call in the whole pipeline is the LLM translation request, and it carries source code, not data rows.

3. LLM call routing

- Every LLM call goes through a single LLMClient (Pydantic AI agent) — there is no other code path that talks to a model.
- Provider and model are selected by **you**, entirely via the LLM_MODEL environment variable (e.g. anthropic:claude-sonnet-4-6, openai:gpt-4o, or an Azure OpenAI deployment). ADC has no default that routes calls through ADC-controlled accounts or infrastructure — you supply your own API key/endpoint.
- Calls originate from the worker container directly to the provider’s API endpoint. ADC has no proxy, logging relay, or intermediary service in that path.
- **Azure OpenAI support exists specifically for customers who need EU data residency and contractual terms consistent with Microsoft’s EU Data Boundary** — pick an Azure OpenAI resource in your preferred region and Rosetta Decode routes to it.
- What is sent per call: a parsed SAS code block (program logic, macro/DATA step/PROC SQL text) plus the pattern context needed for translation. What is **not** sent: reference CSVs, sas7bdat contents, or any row-level data — those are consumed only by the ComputeBackend, which has no network path to the LLM.

4. Layered security architecture

Rosetta Decode’s LLM integration follows the same **defense-in-depth** design ADC applies across its GenAI engagements: no single control is trusted to catch every threat, and controls are layered around the model rather than bolted onto it.

Layer	What it does here	Owner
Auth & authorization (outermost)	Access to the deployment (API, UI, job data) is gated by your own network/identity controls — the product ships with no public endpoint of its own.	Your platform/ops

Layer	What it does here	Owner
Data security & governance	Job data, generated code, and reconciliation reports stay in your PostgreSQL instance; input_hash gives every job a reproducible, auditable fingerprint.	Your platform/ops
User experience / process	Every generated file carries # SAS: <file>:<line> provenance, so a human reviews AI output against its exact source before it's trusted.	Your team
Meta-prompt & grounding	The LLMClient system prompt constrains the model to structured, typed translation output (Pydantic AI schema) rather than free-form generation — reducing prompt-injection and off-task drift.	ADC (product)
Safety systems	Untranslatable or ambiguous constructs are preserved verbatim as # SAS–UNTRANSLATABLE: <reason> rather than silently guessed — no hidden fallback logic.	ADC (product)
AI model (innermost)	The LLM itself is a translation component, never the sole gate on output — it never has autonomous access to your data, only to the code text it's asked to translate.	LLM provider

This mirrors the model used in ADC’s GenAI security engagements: threats are addressed at the layer where they can actually be exploited (application vs. platform vs. usage), not just once at the perimeter.

5. Threat model coverage

Rosetta Decode’s GenAI integration is assessed against the same threat catalog ADC uses across its GenAI security work:

- **OWASP Top 10 for LLM Applications (2025)** — prompt injection, insecure output handling, training data poisoning, model DoS, supply-chain vulnerabilities, sensitive information disclosure, insecure plugin design, excessive agency, overreliance, model theft.
- **MITRE ATLAS** (Adversarial Threat Landscape for AI Systems) — machine-learning-specific attack techniques (model theft, inference attacks, data reconstruction).
- **Microsoft AI Vulnerability Classification** — platform-level risks (privacy attacks, DoS, insufficient logging/auditability, lifecycle governance).

Applied to Rosetta Decode specifically:

- **Prompt injection risk is structurally limited** — the only untrusted input reaching the model is customer SAS source text, and output is constrained to a typed schema, not free text executed downstream without review.
 - **No row-level data exposure to the LLM** — the architectural separation between LLMClient (code only) and ComputeBackend (data only) means there is no path for customer data rows to reach the model, by design, not by policy.
 - **Determinism and auditability** — identical SAS input always produces identical Python output (input_hash), and every generated line is traceable to its SAS source, giving you a complete audit trail for every migration.
 - **No silent failure modes** — untranslatable constructs and LLM failures are surfaced explicitly (SAS-UNTRANSLATABLE, job status=failed), never masked by a fallback that could hide an incorrect translation.
-

6. GDPR compliance posture

- **You remain the data controller** for all SAS code and any data referenced during migration. Rosetta Decode is deployed inside your environment; ADC does not host, access, or process your data as part of running the tool.
- **Data minimization by design** — only code text is sent externally (to the LLM provider of your choice); reference datasets and any personal data they might contain are never transmitted to a third party by the tool.
- **Provider choice enables an EU-compliant processing chain** — choosing Azure OpenAI (or another EU-hosted provider) lets you keep the one external hop within your existing data processing agreements and the Microsoft EU Data Boundary, rather than accepting a fixed ADC-selected provider.
- **No ADC sub-processor role** — because ADC does not operate the runtime, there is no ADC entry required in your Article 30 records or sub-processor list for the tool itself. Your existing agreement with your chosen LLM provider governs that one external call.
- **Provenance supports Article 30 accountability** — every generated line is tagged back to its SAS source, and every job is content-hashed, giving you ready-made documentation of what was processed and how.

Note: this is a description of the technical architecture, not a legal opinion. Your DPO/legal team should confirm this posture against your specific data processing agreements and the personal data (if any) present in your SAS estate before an enterprise pilot.

7. Data retention

- Job records (uploaded file contents, generated Python code, reconciliation reports) are stored in **your** PostgreSQL instance (jobs table), under your retention and backup policy — Rosetta Decode imposes no default expiry today.
- No copy of your SAS code, generated output, or job metadata is retained by ADC, because ADC does not operate the runtime that stores it.
- Retention at the LLM provider is governed by the data processing terms of the provider you select (Anthropic, OpenAI, or Azure OpenAI) — most enterprise API tiers (including Azure OpenAI) do not use API inputs for model training and apply short-lived operational retention only. Confirm current terms with your chosen provider before a pilot.
- **Recommendation for pilots:** agree retention/purge policy for the jobs table up front (e.g., delete job rows N days after handoff) as part of pilot scoping — this is a deployment configuration choice, not a product constraint.

8. Deployment & region

- Rosetta Decode has no fixed hosting location — it runs wherever you deploy the Docker Compose stack: on-prem, in a VM, or in your Azure subscription.
 - For customers pairing with Databricks, execution of reconciliation workloads happens in **your** Databricks workspace/region — ADC does not operate a shared Databricks environment.
 - For the LLM hop specifically, region is determined by the endpoint you configure: an Azure OpenAI resource can be provisioned in an EU region (e.g., West Europe / Sweden Central) to keep that one external call within the EU.
 - There is currently no ADC-hosted “cloud” tier of Rosetta Decode — every pilot to date is a self-hosted deployment in the customer’s own environment or Databricks workspace.
-

9. Talking points for pilot conversations

1. “Your SAS code and data stay in your environment — we deploy the tool to you, we don’t pull your code to us.”
2. “The only thing that leaves your network is the code snippet being translated, sent to the LLM provider you choose — not us.”
3. “Your reference data is never sent to a model — reconciliation runs locally or in your own Databricks workspace.”
4. “Pick Azure OpenAI if EU data residency for that one LLM call is a hard requirement.”
5. “Every line of generated code is traceable to its SAS source — nothing is a black box.”